LOSING MY MARBLES!

"Flick Good, Flick for all the marbles.."

KEITH ASHLY DOMINGO
FakeThird

ADRIEL NEYRO CARAIG
iskwipi

Version 7

Abstract

Currently just a game about marbles, deck building, messing around, and having fun.

Contents

1 Main Concept #
2 Core Gameplay Loop #
3 Player Goals #
4 Game Elements #
5 Game Mechanics #
6 Game Systems #
7 Genre & Inspirations #
8 Feasibility & Risks #
9 Scope Management #
10 Mockup Design #
11 Controls Schema #
12 Prototype Learning #
13 Updated Scope #
14 Architecture Overview #
15 Technical Debt Log #
16 Alpha Goals #
17 Systems Documentation #
18 Asset Pipeline #
19 Playtesting Feedback #
20 Beta Roadmap #
21 Feature Lock Status #
22 Known Issues #
23 Final Week Plan #

24 Postmortem #
25 Architecture Reflection #
26 Code Statistics #
27 Complete Credits #

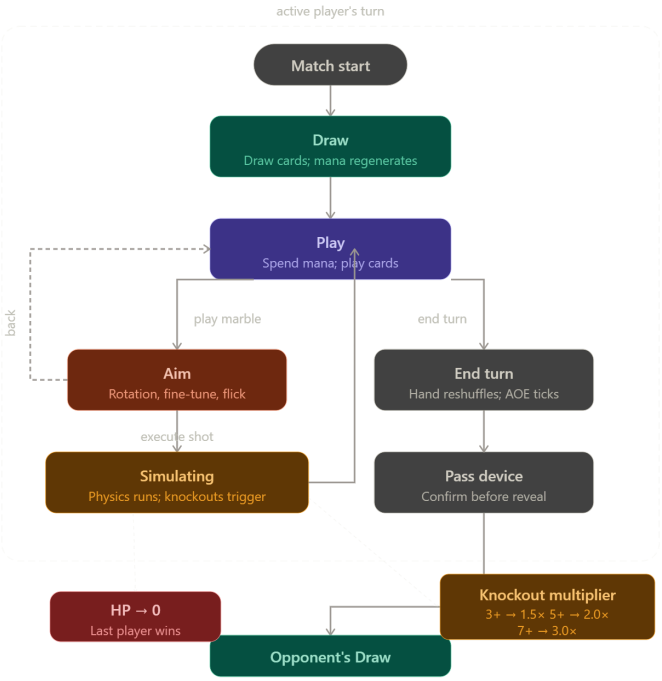
I. Main Concept

Title: **Losing my Marbles!**
Tagline: *"Flick Good, Flick for all the marbles.."*
Hook: A **tactical**, **simulated physics**, **deck builder** reimagining of the Filipino game *'Holen'* that blends precise marble aiming with creative card strategy, where every trick causes mayhem and every shot causes chaos!

II. Core Gameplay Loop

a) **DRAW** → Players draw cards from their private deck; mana regenerates automatically based on the character's mana stat.
b) **PLAY** → The player spends mana to play cards that alter the field or prepare a marble for the upcoming shot. From here, the player chooses one of two branches:
   i) Advance to AIM to take a shot.
   ii) End their turn voluntarily (**END TURN**).
c) **AIM** → The player sets Map Rotation, Fine-Tune Angle, and Flick Slider. A trajectory preview is rendered. The player executes the shot to advance to **SIMULATING**.
d) **SIMULATING** → The player **ends their turn** while the cards they did not use are reshuffled back to their deck and any remaining mana is discarded
e) **END TURN** → Reached only from **PLAY** via Branch B. Unplayed cards reshuffle; mana is discarded; AOE durations tick down. The next player's **DRAW** begins.

Note: Bullets b and c can be done multiple times per turn



Frame A. Mockup In Game GUI by Keith Ashly Domingo

III. Player Goals

A. Before a match:

- Choose a character whose Mana and Power stats fit the intended strategy.
- Build a private deck (any card type) with a clear win condition.
- Build a public marble deck that populates the shared field pool at match start.

B. During a match:

- Knock opposing marbles off the field to trigger their passive effects.
- Protect your own health from reaching zero.
- Manipulate field physics using Terrain and AoE cards to set up chain shots.

Win Condition: Last player with health remaining wins.

IV. Game Elements

A. The Map

- The main playing field where marbles are placed and shot at
- Has variables that affect how shots travel and how marbles interact with each other
- Properties: shape, depth, surface friction, wind resistance

B. The Character

- The unit in which the player plays as
- Has exclusive cards that the player can add to their deck
- Properties: health, mana, power, exclusive cards

C. The Deck

- The set from which cards are drawn during a match
- Is set up by the player before a match using cards from their collection
- Refreshes once all cards have been used
- Split into two: public and private

i. Public Deck

- Is only made up of marble cards
- At the start of a match, the public decks of all players are combined into a single pool of cards from which the marbles are drawn to put into the playing field every time the field is empty

ii. Private Deck

- Can be made up of all card types
- Is the deck from which the player draws cards per turn

D. The Cards

- The main unit used for building a strategy
- Has a variety of effects to either provide advantage or disrupt opponents
- Categorized into four: marble, power-up, trick, or terrain card

E. The Marble Cards

- The main unit used for executing shots
- Interacts with the map and other marbles accordingly
- Behaves differently when in the playing field and when being played for a shot
- Properties: weight, friction, passive effect, active effect
  - **i. When in player's hand**
    - The active effect of a marble card is applied to the shot the player is currently preparing
  - **ii. When in the playing field**
    - The passive effect of a marble card is triggered when that marble is knocked off the playing field either by a shot or by other card effects

F. The Non-Marble Cards

- The units that enables the player to build powerful strategies
- Properties: mana requirement
  - **i. Power-up**
    - Affects the marble that the player currently prepared or the player stats themselves
  - **ii. Trick**
    - Affects either the marbles currently on the playing field, any of the players' decks, enemy stats and marble, shooting conditions, and everything not in power-up.
  - **iii. Terrain**
    - Affects the current map condition which affects all players
    - Only one terrain card can be active at a time

V. Game Mechanics

A. Match Initialization

- A random map is selected, defining the playing field and establishing the environment conditions using the surface and wind properties of the map
- The public decks of all players are combined into a single pool of cards and the map is set up by drawing a certain number of marbles from the pool and randomly placing them on the playing field
- The turn order for the players will be decided randomly

B. Player Turn

- The player draws a certain number of cards from their deck
- The player generates a certain amount of mana

- The player can play terrain cards which will override the existing terrain, since only one terrain card can be active at once
- The player can play a single marble card that changes the properties of the marble for the current shot being prepared
- The player can play any number power-up and trick cards as long as they have the resources to meet the requirements of those cards
- The player can switch to aiming mode to display and adjust the predicted trajectory of the current shot then finally execute the prepared shot
- The shot is then simulated to produce an outcome of which marbles stayed in the playing field and which marbles were knocked off, then triggering the effects of the latter to wherever they apply
- If there are no more marbles left on the playing field, the game will draw marbles from the pool created in the match initialization to refill the playing field
- If there are no more cards on the player's deck, the game will reshuffle all of the player's consumed cards to be used for that player's next draws
- The player can perform as many shots as they have the resources to do so
- The player can end their turn whenever they desire
- After ending their turn, the player reshuffles their hand back to their deck and discards any remaining mana, then the next player will be able to start their turn

C. Multiplier

Tracks how many marbles exit the field when a marble has been shot:

- 0–2 knockouts → 1.0×
- 3–4 knockouts → 1.5×
- 5–6 knockouts → 2.0×
- 7+ knockouts → 3.0×

Multiplier scales effect values via effect.duplicate() — original .tres resources are never mutated. Resets at the start of each new marble shot.

D. Win Condition

- A player will be eliminated from the match if their health falls down to zero
- A player wins the match by being the last player alive

VI. Game Systems

A. Main UI

- Ensures players have access to their resources while achieving the vibe of the game

B. Physics Engine

- Simulates shot-environment interactions and marble collisions

C. State Machine

- Govern game phases and player turn orders

D. Effect Handler
- Handles marble and card effects for all game elements

E. Inventory Manager
- Manages card collection and deck building

VII. Genre & Inspirations

Primary Genre: **Hybrid Deck Builder**
Categories: Card Games, Simulated Physics, Turn-Based Strategy
Reference Works:

- **Kabuto Park:** The overall vibe and theme and main battle UI
- **Slay the Spire:** The use of selectable heroes with specific cards only available to them, mana usage for card use, and the refresh of the hand per turn unless a card says otherwise.
- **Potionomics:** Cards relating only to certain characters and the deck building aspects, and card design.
- **Peglin:** The mechanic where orb varieties have inherent characteristics and behave differently with their surroundings based established game physics
- **Balatro:** The mechanic where a good setup results into a huge payoff, which is achieved through creative strategies and exciting win affirmations for player satisfaction
- **Pokemon TCGP:** The deck-building system and strategy balancing through proper card variance that produces countless game tactics.

VIII. Feasibility & Risks

A. Feasibility Check
- By Week 5, there should be a working game loop that demonstrates marble shot simulation and deck management, with some game mechanics already implemented and a sample set of playing cards

B. Technical Risks:

i. Physics Implementation
- Simulating environment and marble physics may get out of hand when accounting for too much entities and variables
- Probability of desynchronization with predicted path travel when calculating due to combination of effects and entities affecting the physics of the marble

ii. Game Balancing

- The gameplay and the cards themselves should feel fair but competitive for all players, which might take a lot of time to get right

iii. Finding Assets

- As the game is pretty unique in its themes, it will be difficult to find quality assets that achieve a fun yet comforting vibe

iv. Smart AI Implementation

- As it will be unfair for the user to be limited to play the game with only having a partner, an implementation of solo play will be required but the challenge will be implementing an AI that the user can compete with.

v. Single Device Turn Based Battle

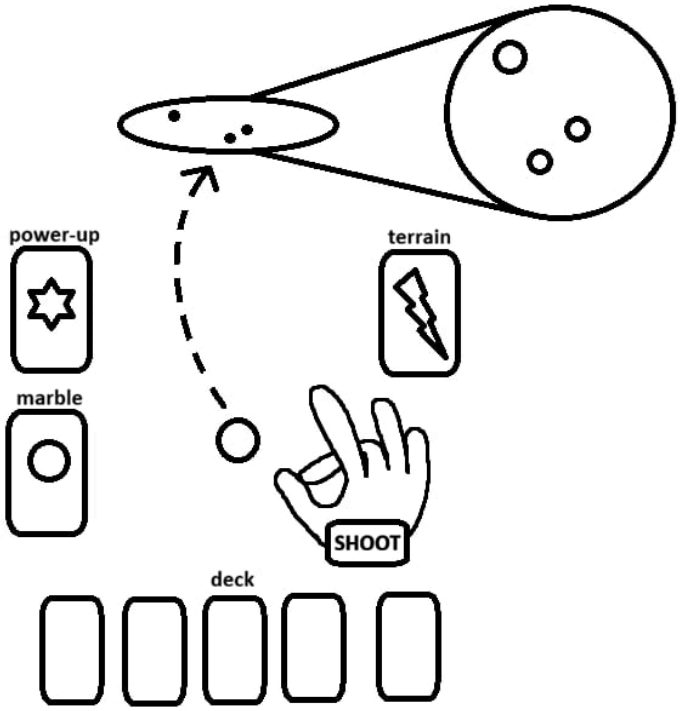
- Due to the current scope the game will be limited to just one device where the problem currently resides is in making sure that the turn based aspect still feels fun and not stale.

IX. Scope Management

1. **Minimum Viable Product**
    a. A demo game presenting a complete implementation of all the game mechanics but with 2 sample decks, 2 sample characters, 1 sample map, and PvP gameplay only
2. **Stretch Goals**
    a. Broad selection of cards, characters, and maps
    b. Cohesive UI/UX that achieves the overall theme
    c. Immersive story and balanced game progression
3. **Maximum Final Product**
    a. An online multiplayer mode to battle other players with cards and characters drawn from packs and loot boxes bought using in-game currency earned by progressing through an immersive offline campaign where the player battles the computer with progressive difficulty
4. **Excluded Features**
    a. 3D simulation for marble physics
    b. World exploration or free roam
    c. Real-time action gameplay

X. Mockup Designs

Design A. Mockup In Game GUI by Keith Ashly Domingo



Design B. Mockup Shooting GUI by Adriel Neyro Caraig

XI. Controls Schema

1) **Input** — Action
2) **Drag card toward center** — Play a card from your hand
3) **Flick Slider** — Set how hard you shoot (0 = softest, 10 = hardest)
4) **Map Rotation buttons (hold)** — Spin the entire field to change shot direction

5) **Fine-Tune buttons (hold)** — Make small angle adjustments before shooting
6) **Aim button** — Switch to aiming mode
7) **Execute button** — Fire the marble
8) **Back button** — Return to card-playing without shooting
9) **End Turn button** — Voluntarily end your turn

The game separates big direction changes (Map Rotation) from small precision tweaks (Fine-Tune) to mirror how a real marble player repositions their body then adjusts their wrist. Dragging a card before playing it makes the cost feel tangible. Button-based input was chosen over free mouse-click aiming so the game works equally well on desktop and on a single shared device passed between players.

XII. Prototype Learnings

1) What worked

The physics felt immediately fun. Marbles bounced, rolled, and knocked each other around in satisfying and unpredictable ways from the very first test and no major overhaul was needed. Cards were also easy to create; because each card is just a data file listing its effects, new cards could be added quickly without touching any game logic. The shot preview was clear and updated smoothly. Pass-and-play worked cleanly, correctly hiding each player's hand and requiring a confirmation tap before anything was revealed.

2) What is being cut or reworked

Online multiplayer during active development is being deferred. The online flow gets stuck at the character selection screen and does not proceed to the match. Rather than delay everything else, online play is moved to the final phase — fixing it requires no rewrites since the underlying architecture already supports it.

Per-marble friction differences are also deferred. All marbles currently slide at the same speed because a global field setting overrides their individual values. Restoring marble-to-marble feel differences is a post-submission tuning task.

3) Technical surprises

The card drag-and-play had a hidden timing issue. The card plugin's confirmation signal only fires after a slow animation finishes, which causes the game to miss card plays entirely. This was resolved by hooking into a different point in the plugin. Exit detection also triggered falsely, marbles were being counted as knocked out the moment they spawned. The fix was to only count marbles that had already been confirmed as inside the field first. On the positive side, offline and online code turned out to share the exact same execution path, meaning no separate offline version of the logic was needed and this saved a significant amount of work.

XIII. Updated Scope

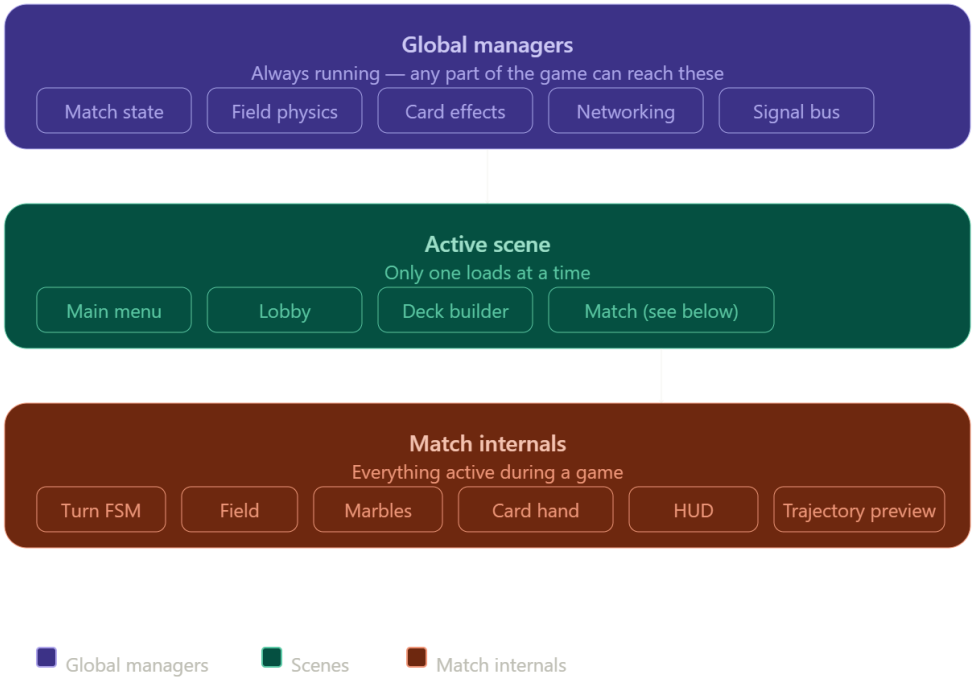
The MVP is a complete two-player offline pass-and-play match with all core systems working:

- **the full turn flow of draw play, aim, shoot, and opponent's turn**
- **two playable characters with different stats**
- **over 20 cards across all five card types**
- **a physics field with friction, gravity, and stacking effects**
- **a knockout chain multiplier scaling up to 3×**
- **deck cycling with a discard pile**
- **a one-marble-per-shot rule enforced with a UI hint.**

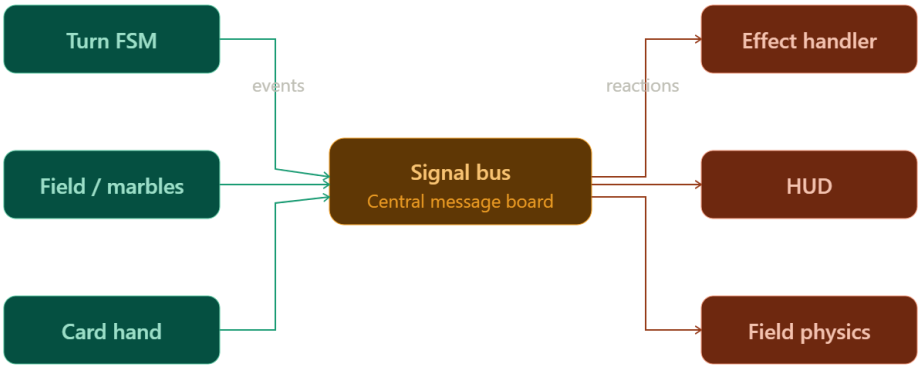
The online multiplayer skeleton is in place but the full online match flow is deferred to the final phase.

What is not in this build: **online multiplayer match flow, automated testing, and per-marble friction differences.**

XIV. Architecture Overview



Frame B. Scene Hierarchy



Frame C. Information flow between systems (Note: No system talks to another directly — everything goes through the bus)

Three design patterns are used throughout. The **Observer pattern** is the Signal Bus, systems broadcast events and others listen without knowing who is listening. The **State Machine** pattern controls the turn; the game always knows exactly which phase it is in and blocks actions that don't belong there. The **Data pattern** covers cards, every card is just a plain file describing its cost and effects, and a separate handler reads and executes them. This means a new card can be created by a designer without touching any game code.

XV. Technical Debt Log

Issue	What it means	Why it was left	Plan
Online match stalls at character select	Both players can choose a character but the game never starts online	Fixing it would have blocked all offline development for weeks	Post-Defense Development
All marbles slide identically	Each marble type has its own friction value stored but a global override wipes them out	Low priority compared to getting core loop working	Post-Defense Development
Card plugin workaround	The plugin fires its "card played" signal too late, so we hook into an earlier internal point	Rewriting the plugin's card behavior from scratch was too costly	Post-Defense Development

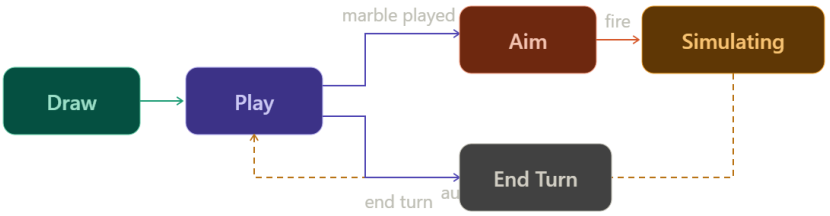
XVI. Alpha Goals

A full match must be playable from start to finish with no crashes. This means both players can draw cards, spend mana, play any of the four card types, aim and shoot, trigger knockout effects and the multiplier chain, cycle through their deck, and pass the device all the way until one player reaches zero health and a winner is declared. Every system must be present and connected, even if not visually polished. All the arts were no longer placeholder and had been exchanged with custom done assets.

XVII. Systems Documentation

The game has four core systems that work together to run a match.

The turn system is a **state machine**: a rulebook that decides what the player is allowed to do at any given moment. It moves through five phases in order: Draw, Play, Aim, Simulating, and End Turn. It enforces the rules automatically, so a player cannot fire a marble during the Play phase or play a card during Simulating. The server always controls this; the opponent's screen just reflects what the server says.



Frame D. Flow of Phases

The card system treats every card as a plain data file listing what it costs and what it does. A separate Effect Handler reads those files and carries out the action. There are eight possible effects: dealing damage, healing, draining or restoring mana, and four ways to modify the physics field. Any card can carry any combination of these.

Card type	When it fires	What it affects
Marble	On play + knockout	The shot itself; opponent on knockout
Power-up	When played	Active player (health, mana)
Trick	When played	Opponent or field

Terrain	When played	Global field (one active at a time)

The physics field runs on a three-layer stack. The base layer is the permanent surface of the map. The terrain layer is whatever terrain card is currently active, new terrain overwrites the old one. The AoE from the Trick layer is a pile of temporary effects that stack on top of each other and count down each turn. Every time any layer changes, the game recalculates the combined result and applies it to all marbles on the field instantly.

There is no progression, leveling, or save system. Character selection happens before each match and does not persist. Deck choices are set in the deck builder and carried into the match, but nothing carries over between sessions.

XVIII. Asset Pipeline

Only two imported credited assets were used in the making of the project, both were free to use in itch.io:
Fonts by Otto Ajala - https://ottoojala.itch.io/pirkkala
Music by - dani maccari - https://dani-maccari.itch.io/jumppack-music-loops

Everything else was made by one of the developers, **Keith Ashly M. Domingo.**

XIX. Playtesting Feedback

No playtesting happened but a couple of comments from other developers and onlookers were taken into account during development:

- “Change the shooting and aiming features to be more similar to pool games rather than just having them as sliders and buttons respectively”.
- “Add a visual mechanic where marbles shot out don't just disappear but are still on the field but made transparent and not interactable”.
- “Make the visuals much more user attentive for an easier time of learning what the game does”.

XX. Beta Roadmap

1) What gets finished in the following weeks:

All remaining gameplay features will be locked and stabilized. This means final card balance tuning, adjusting damage values and mana costs so no single card or combo feels overwhelming, and restoring per-marble friction so each marble type feels physically distinct. Placeholder visuals will be swapped for real art assets as they become available, with marble sprites first since those are the most visible during play. Sound effects will be reviewed and finalized. The pass-and-play flow will receive a polish pass to feel smoother between turns.

2) **What is being cut for time:**

Online multiplayer will not be in the beta. The character-select-to-match transition remains broken in online mode and fixing it would consume the entire remaining schedule. The skeleton is in place and the architecture supports it, so it is preserved for a potential post-submission update.

Automated testing and strict code-quality enforcement are also deferred. The game has been stable enough through manual testing that this is a reasonable cut.

3) **Polish priorities in order:**

First, gameplay stability: no softlocks or crashes through a full match.
Second, card balance so the game feels fair and competitive.
Third, sound and screen-feel polish.
Lastly, UI clarity, by that i mean button labels, turn indicators, and the HUD readability.

XXI. Feature Lock Status

Feature	Status	Why
Full turn loop (Draw → Play → Aim → Simulate)	In	Core mechanic
3 playable characters with different stats	In	MVP requirement
20+ cards across all 4 card types	In	MVP requirement
Knockout chain multiplier (up to 3×)	In	Core mechanic
Additive physics field layering	In	Core mechanic
Deck cycling with discard pile	In	Core mechanic
Pass-and-play with device handoff screen	In	Offline multiplayer
Trajectory preview during aiming	In	Core mechanic
Deck builder before match	In	MVP requirement

LAN multiplayer skeleton	In	Architecture ready
Online multiplayer match flow	Cut	Blocks all other work
Per-marble friction differences	Cut	Low impact, low time
Automated test suite	Cut	Time vs. reward
Stickiness physics property	Cut	Field exists, unused

XXII. Known Issues

1) **Bugs that will not be fixed before Week 16:**

Online multiplayer is not complete. Both players can reach the character select screen but the match never starts. The underlying code is correct but the timing of when both players confirm their selections does not trigger the transition reliably. This is a known, isolated issue and does not affect offline play in any way.

Per-marble friction is non-functional. Each marble type stores its own friction value but a global field setting overwrites them all, making every marble feel the same on the surface. The values are ready to use the moment the override is removed, this is a one-line fix that was deprioritized in favor of stability.

The card plugin workaround is fragile. The game hooks into an internal point of the card plugin that was not designed to be used this way. If the plugin is updated, this hook may break. The risk is low since the plugin is not actively maintained, but it is worth noting.

2) **Performance concerns:**

No significant lag spikes have been observed during testing. The physics simulation runs only during the Simulating phase and settles quickly. The main risk is a match with many marbles on the field simultaneously but this has not been stress-tested beyond normal play conditions.

3) **Edge cases:**

If a player's draw deck and discard pile are both empty at the same time, the draw phase has nothing to pull from. This is theoretically possible in a very long match but has not been triggered in any playtest session. A guard exists in the code but has not been tested against a real occurrence.

If both players reach zero health in the same simulation phase like for example, two marbles knock each other out simultaneously and both trigger lethal damage, the win condition

check currently declares the active player the winner. This is an untested scenario and the correct behavior has not been formally decided.

XXIII. Final Week Plan

1) **Bug fixing priorities in order:**

First, verify the win condition edge case above and decide the intended behavior. Second, test the empty deck guard under controlled conditions and confirm it resolves cleanly. Third, run a full match from character select to win screen on a clean export build and catch any remaining transition issues.

2) **Polish tasks:**

Review all sound effects for volume consistency, shots, card plays, and knockouts should feel proportionally weighted. Add a brief screen shake on high-impact collisions. Ensure the HUD is readable at a glance: current phase, both players' health and mana, and whose turn it is must all be immediately clear without reading any label twice. Review the pass-and-play handoff screen for clarity and the prompt to pass the device should be impossible to miss.

3) **Presentation preparation:**

The live demo should show one complete match from deck building through to a win, hitting at least one knockout chain to demonstrate the multiplier while also explaining the main mechanics and elements of the game.

XXIV. Postmortem

1) **What went well:**

Overall, we think that we have established a strong foundation for the base game. For such a limited time frame, we have managed to fully design the cards and marbles game concept, conceptualize an initial set of characters and cards, implement a stable physics control and simulation, build a robust card game state chart, integrate a proper card framework, create working resource library, character selector, and deck builder interfaces, and set up a prototype effect handler equipped with initial commands already implemented. Additionally, our decision of moving from implementation-from-scratch to plugin-integration saved us a lot of time and provided us with room to focus more on the core logic of our game.

2) **What would you change:**

In the middle of the semester, building our Machine Problem 2 made us realize that we can also integrate online multiplayer in our game. This however turns out to be a bad idea and ultimately makes our scope almost impossible to achieve within the timeline. We think that we should not have refactored in the middle of development to integrate the multiplayer

feature and instead focus on implementing the entire game with pass-and-play. This would have saved us a lot of time in developing and testing.

3) **Key learnings:**

We have learned that properly setting a strict scope will lead to significantly less time and effort wastage. We have always been ambitious in all of the games we have developed. Yet, we have realized that we are still limited with what we can do and what resources we only have. For our future projects, this is one thing we will sure take note of right from the start.

XXV. Architecture Reflection

1) **Which design patterns saved you**:

The most important design patterns we employed are: **state chart** (turn logic), **observer pattern** (signal bus), **data pattern** (resource library), **command pattern** (effect handler), while some others contribute to specific applications.

2) **Where did your architecture break down:**

Our architecture broke down when it came to integrating the assets. This is due to the nature of our workflow, but its effect for this project increased proportionally with the sheer complexity of the system we are trying to implement. Additionally, the nature of a card game also tends to break the current game architecture due to the system always needing to adapt the new card mechanics.

3) **How would you architect it differently:**

For future projects, we think extensive research should be done first to study how existing systems similar to what we are trying to implement work. In this particular game, we think we can implement our own physics engine and card framework built on top of existing native Godot resources in order to create a seamless project architecture.

XXVI. Code Statistics

Lines of code - We have estimated that our codebase have about **3,500** lines of code

Number of commits - As of writing, we have a total of **119** commits with descriptive prefixes and commit messages

Refactors performed - Overall, we estimate that **8 to 10** refactors based on the scope and definition of a refactor, mostly on the design decisions for the final product

XXVII. Complete Credits

1) **All asset attributions:**
   - Fonts by Otto Ajala - https://ottoojala.itch.io/pirkkala
   - Music by dani maccari - https://dani-maccari.itch.io/jumppack-music-loops

- Game Art by Keith Ashly M. Domingo

2) **Third-party code/plugins:**
   - Godot State Charts by derkork - https://github.com/derkork/godot-statecharts
   - Godot Card Framework by chun92 - https://github.com/chun92/card-framework
   - Phantom Camera by ramokz - https://github.com/ramokz/phantom-camera